Honeypot IDS – Pool test report

Generated: 2026-10-05 11:55:14
Target: ids (local) (https://localhost)
Exploits: A: CVE-2024-2387, B: CVE-2025-47577, C: GENERIC-WP-003, A: CVE-2024-1071, B: CVE-2025-4403, C: GENERIC-WP-002
Windows: 3

Result

SHARED EXPECTED **Pools are shared, as expected under resource pressure**

- Pool 2 is shared by Attacker A, Attacker C.
- This is the designed fallback: no free pool was left, so new attackers were assigned to the ready pools round-robin.
- Pool 2 (owned by window C) also served window(s) A.
- Pool 2 (owned by window C) also served window(s) A.

What is being tested

Attackers are told apart by session, not by address. Each test window has its own cookie jar and its own User-Agent and language, so the proxy sees three separate attackers. By default every malicious session is given a honeypot pool **of its own**: the router hands a new attacker a free ready pool no other session owns, and pool_manager builds a spare after each assignment. Two attackers only share a pool when none is free – growth is capped because the host is short of memory, CPU or disk, or every ready pool is already owned – and the router then reuses the ready pools in strict round-robin order.

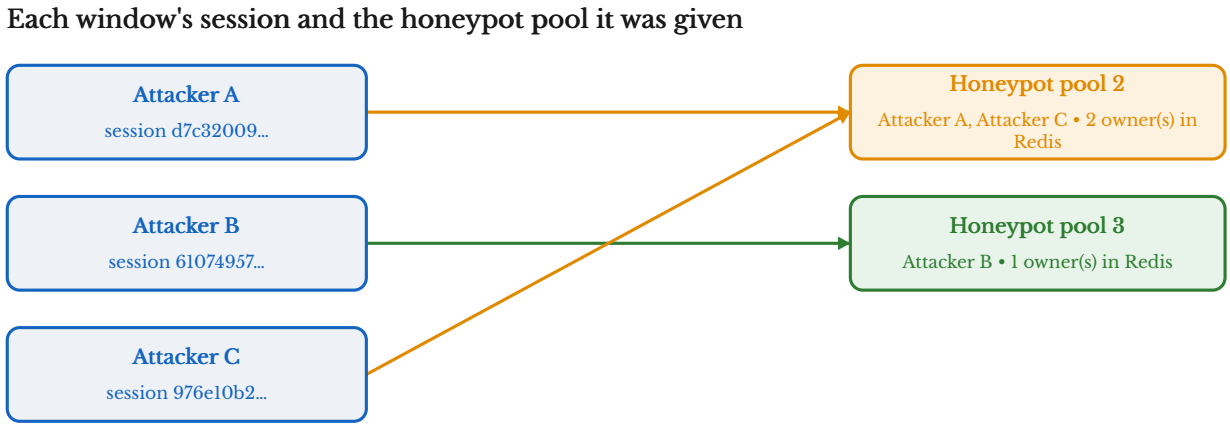


Figure 1. One arrow per test window, from its session to the honeypot pool the router bound it to. Green: the pool belongs to that window alone. Orange: shared under resource pressure (designed). Red: shared or merged without a reason. Grey: not bound.

Windows

Window	Session	Honeypot pool	Language	User-Agent (tail)
Attacker A	d7c32009…	pool 2	en-US,en;q=0.9	6.11.2 Chrome/140.0.0.0 Safari/537.36 PoolTest/A
Attacker B	61074957…	pool 3	de-DE,de;q=0.9	6.11.2 Chrome/140.0.0.0 Safari/537.36

				PoolTest/B
Attacker C	976e10b2…	pool 2	fr-FR,fr;q=0.9	6.11.2 Chrome/140.0.0.0 Safari/537.36 PoolTest/C

Cart persistence

A visitor diverted to a honeypot is served by a different database, so WooCommerce's own cart session does not exist there. The storefront therefore carries the cart in a cookie and rebuilds it on whichever instance serves the next request. Each window filled its cart on production; the cart was then read back after every step, through the same cookies and routing as the window's own traffic.

Step	Attacker A	Attacker B	Attacker C
All windows open the shop	no cart yet on production	no cart yet on production	no cart yet on production
Every window fills its cart	same cart on production	same cart on production	same cart on production
Attacker A attacks with CVE-2024-2387	same cart on pool 2	same cart on production	same cart on production
Attacker B attacks with CVE-2025-47577	same cart on pool 2	same cart on pool 3	same cart on production
Attacker C attacks with GENERIC-WP-003	same cart on pool 2	same cart on pool 3	same cart on pool 2
All windows run a second exploit	same cart on pool 2	same cart on pool 3	same cart on pool 2
Every window opens its cart	same cart on pool 2	same cart on pool 3	same cart on pool 2

Every cart stayed exactly as filled, on production and in every honeypot pool.

Exploits run per window

Each window runs a different exploit, picked from the ones a live probe showed divert within three attempts. If a chosen exploit still did not divert in three attempts, the next candidate is used instead.

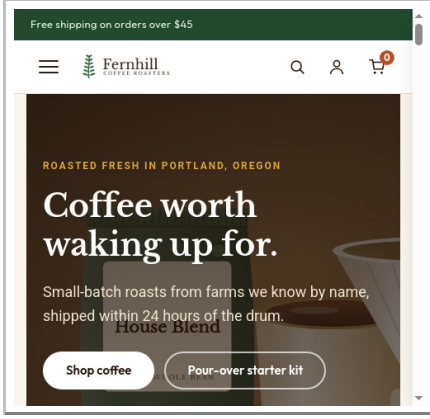
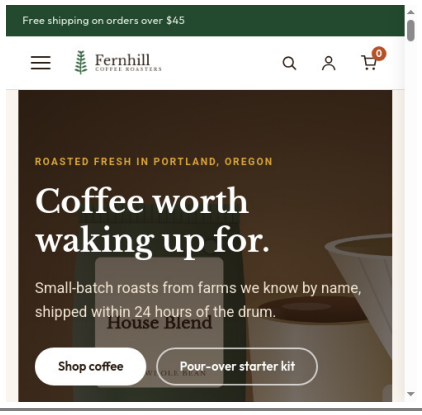
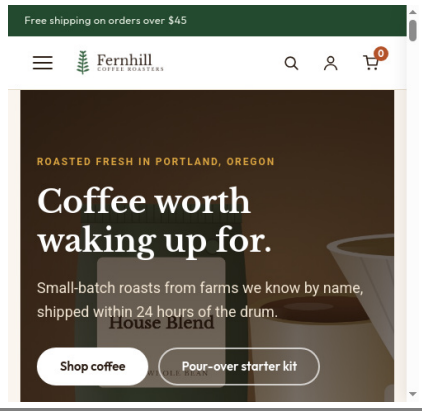
Window	Exploit	Name	Routing	Pool
A	CVE-2024-2387	Advanced Form Integration SQL injection	diverted after 1	pool 2
B	CVE-2025-47577	Gift Voucher file upload RCE	diverted after 1	pool 3
C	GENERIC-WP-003	User enumeration via ?author=	diverted after 1	pool 2
A	CVE-2024-1071	Ultimate Member SQL injection	diverted after 1	pool 2
B	CVE-2025-4403	Drag & Drop file upload MIME-type bypass	diverted after 1	pool 3
C	GENERIC-WP-002	Installer file access	diverted after 1	pool 2

Step by step

All three windows move in every step. An attacker's pool must appear in their own step, differ from the earlier ones, and never change afterwards.

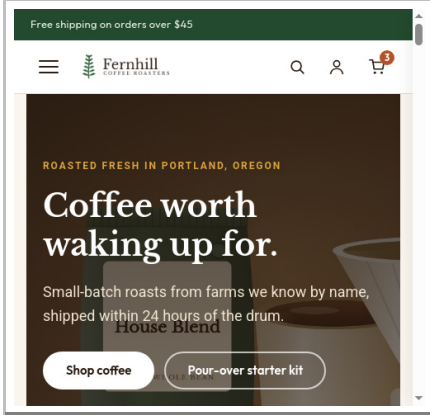
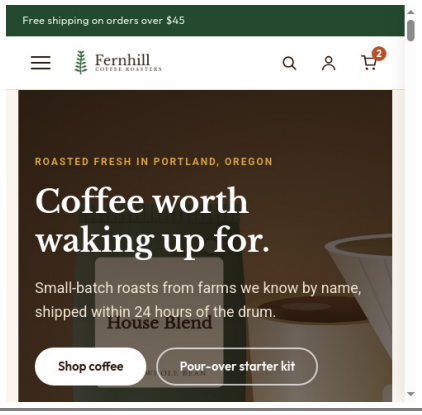
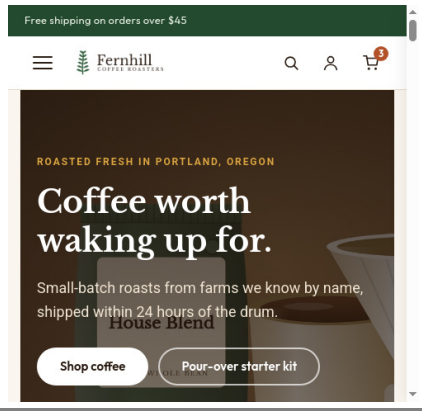
Step 1: All windows open the shop

Three new sessions, all routed to production.

Attacker A	Attacker B	Attacker C
Browses the shop	Browses the shop	Browses the shop
PRODUCTION no pool • score 0	PRODUCTION no pool • score 0	PRODUCTION no pool • score 0
cart empty	cart empty	cart empty
		

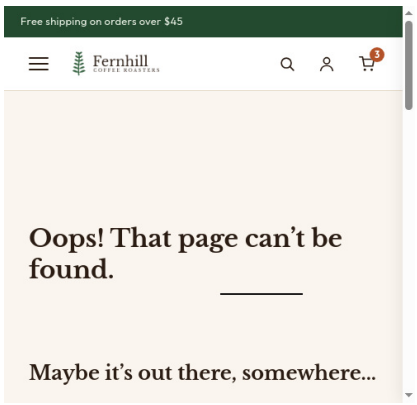
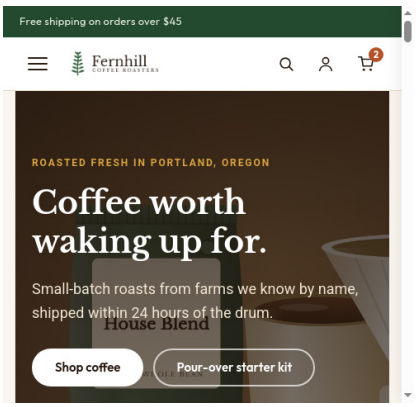
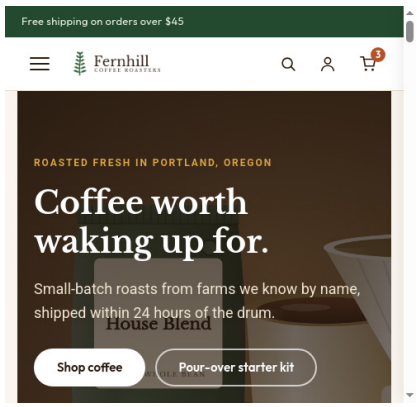
Step 2: Every window fills its cart

Each window adds its own products. These carts are the baseline: they must look exactly the same after the session is routed to a honeypot pool.

Attacker A	Attacker B	Attacker C
Adds ceramic-pour-over-dripper × 1, paper-filters-size-02 × 2	Adds hand-burr-coffee-grinder × 1, stainless-milk-pitcher × 1	Adds enamel-camp-mug × 2, fernhill-canvas-tote × 1
PRODUCTION no pool • score 0	PRODUCTION no pool • score 0	PRODUCTION no pool • score 0
$46.00 Ceramic Pour-Over Dripper × 1 Paper Filters, Size 02 (100 ct) × 2	**$113.00** Hand Burr Coffee Grinder × 1 Stainless Milk Pitcher 12 oz × 1	**$60.00** Enamel Camp Mug × 2 Fernhill Canvas Tote × 1
		

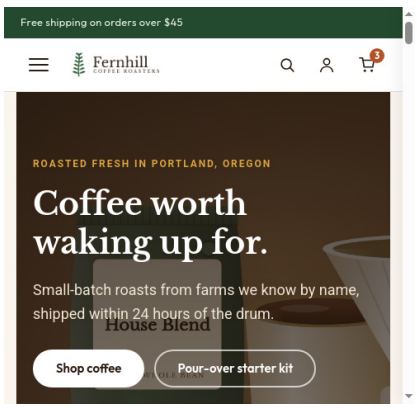
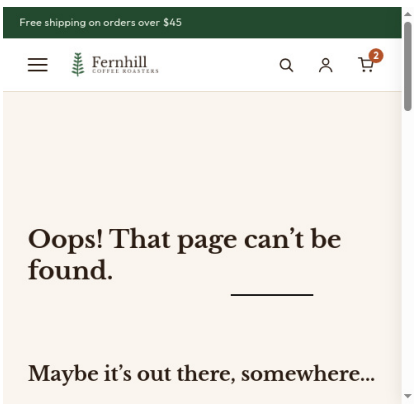
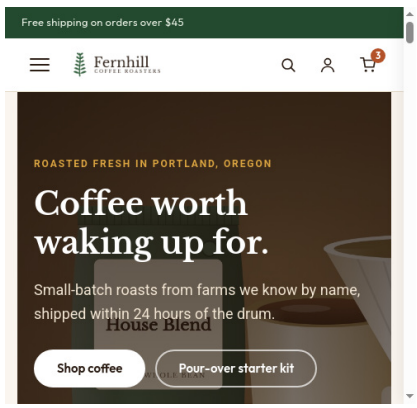
Step 3: Attacker A attacks with CVE-2024-2387

Advanced Form Integration SQL injection. The session must be bound to a pool no other window has, and the others must stay on production.

Attacker A	Attacker B	Attacker C
Opens CVE-2024-2387 (1 attempt(s))	Browses the shop	Browses the shop
HONEYPOT pool 2 • score 100	PRODUCTION no pool • score 0	PRODUCTION no pool • score 0
$46.00 Ceramic Pour-Over Dripper × 1 Paper Filters, Size 02 (100 ct) × 2	**$113.00** Hand Burr Coffee Grinder × 1 Stainless Milk Pitcher 12 oz × 1	**$60.00** Enamel Camp Mug × 2 Fernhill Canvas Tote × 1
		

Step 4: Attacker B attacks with CVE-2025-47577

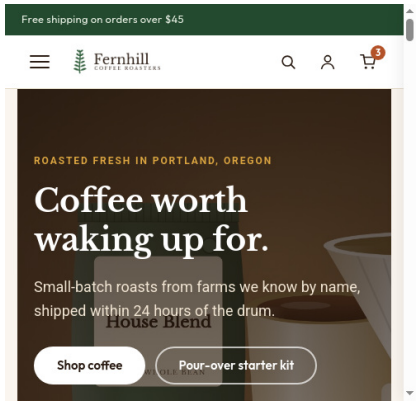
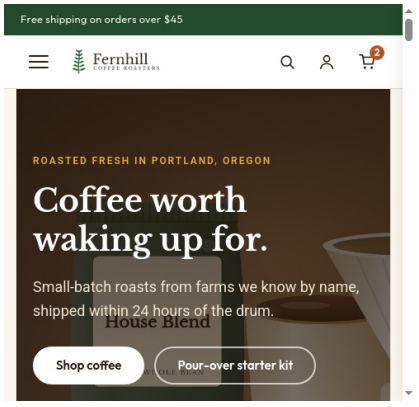
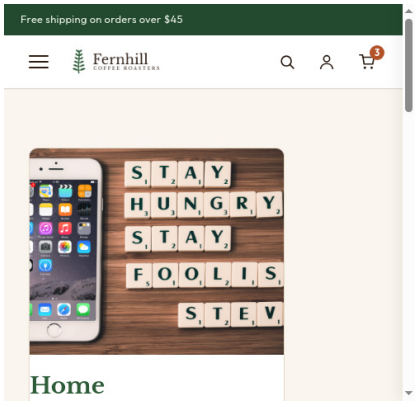
Gift Voucher file upload RCE. The session must be bound to a pool no other window has, and the others must stay on production.

Attacker A	Attacker B	Attacker C
Browses the shop	Opens CVE-2025-47577 (1 attempt(s))	Browses the shop
HONEYPOT pool 2 • score 100	HONEYPOT pool 3 • score 95	PRODUCTION no pool • score 0
$46.00 Ceramic Pour-Over Dripper × 1 Paper Filters, Size 02 (100 ct) × 2	**$113.00** Hand Burr Coffee Grinder × 1 Stainless Milk Pitcher 12 oz × 1	**$60.00** Enamel Camp Mug × 2 Fernhill Canvas Tote × 1
		

Step 5: Attacker C attacks with GENERIC-WP-003

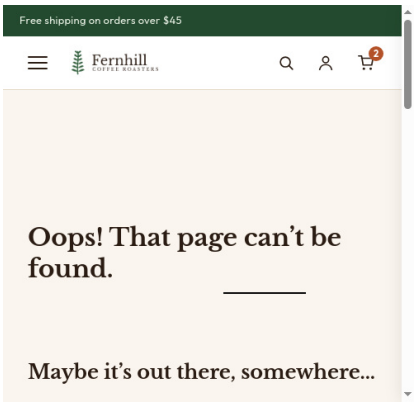
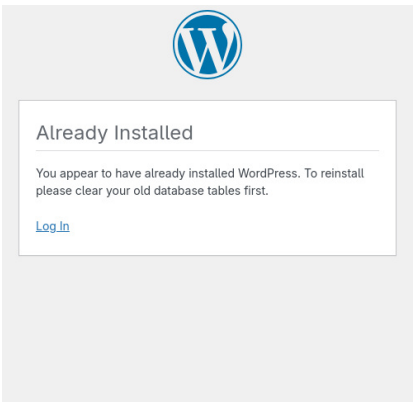
User enumeration via ?author=. The session must be bound to a pool no other window has, and the others must stay on production.

Attacker A	Attacker B	Attacker C
Browses the shop	Browses the shop	Opens GENERIC-WP-003 (1 attempt(s))
HONEYPOT pool 2 • score 100	HONEYPOT pool 3 • score 100	HONEYPOT pool 2 • score 100

$46.00 Ceramic Pour-Over Dripper × 1 Paper Filters, Size 02 (100 ct) × 2	**$113.00** Hand Burr Coffee Grinder × 1 Stainless Milk Pitcher 12 oz × 1	**$60.00** Enamel Camp Mug × 2 Fernhill Canvas Tote × 1
		

Step 6: All windows run a second exploit

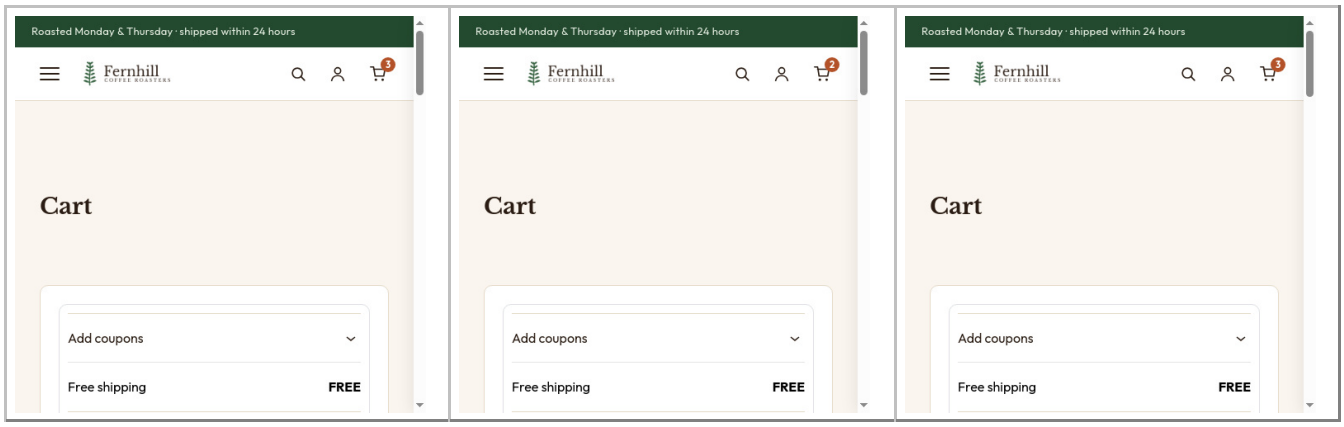
Sticky binding: each second exploit must land in the pool the window already owns, nobody changes pool, nobody is merged.

Attacker A	Attacker B	Attacker C
Opens CVE-2024-1071	Opens CVE-2025-4403	Opens GENERIC-WP-002
HONEYPOT pool 2 • score 100	HONEYPOT pool 3 • score 100	HONEYPOT pool 2 • score 100
$46.00 Ceramic Pour-Over Dripper × 1 Paper Filters, Size 02 (100 ct) × 2	**$113.00** Hand Burr Coffee Grinder × 1 Stainless Milk Pitcher 12 oz × 1	**$60.00** Enamel Camp Mug × 2 Fernhill Canvas Tote × 1
0		

Step 7: Every window opens its cart

The cart page, served by each window's honeypot pool, must show the same products and quantities that were added on production.

Attacker A	Attacker B	Attacker C
Opens the cart page	Opens the cart page	Opens the cart page
HONEYPOT pool 2 • score 100	HONEYPOT pool 3 • score 100	HONEYPOT pool 2 • score 100
$46.00 Ceramic Pour-Over Dripper × 1 Paper Filters, Size 02 (100 ct) × 2	**$113.00** Hand Burr Coffee Grinder × 1 Stainless Milk Pitcher 12 oz × 1	**$60.00** Enamel Camp Mug × 2 Fernhill Canvas Tote × 1



What each honeypot container saw and changed

Read from each pool's own eshop and database containers since the run started (2026-10-05T09:54:41Z). Requests are matched to windows by their User-Agent tag, so a pool should show its owner's exploits and nobody else's. A pool no window owns is shown for contrast.

pool 2 window C

Windows seen	A, C (17 requests)
Requests received	• A GET /?page=advanced-form-integration-log&integration_id=1%27%20UNION%20SELECT%20user_login%2Cu → 404 • A GET /wp-json/wc/store/v1/cart → 200 • A GET / → 200 • C GET /?author=1 → 301 • C GET /author/admin/ → 200 • C GET /wp-json/wc/store/v1/cart → 200 • C GET /wp-admin/install.php → 200 • A GET /wp-admin/admin-ajax.php?action=um_get_members&directory_id=1%27%20UNION%20SELECT%20user_l → 400
Files changed	• no files written
Database activity	questions +3161, select +2995, insert +20, update +8, delete +8

pool 2 window C

Windows seen	A, C (17 requests)
Requests received	• A GET /?page=advanced-form-integration-log&integration_id=1%27%20UNION%20SELECT%20user_login%2Cu → 404 • A GET /wp-json/wc/store/v1/cart → 200 • A GET / → 200 • C GET /?author=1 → 301 • C GET /author/admin/ → 200 • C GET /wp-json/wc/store/v1/cart → 200 • C GET /wp-admin/install.php → 200 • A GET /wp-admin/admin-ajax.php?action=um_get_members&directory_id=1%27%20UNION%20SELECT%20user_l → 400

Files changed	• no files written
Database activity	questions +3164, select +2996, insert +20, update +8, delete +8

pool 3 window B

Windows seen	B (8 requests)
Requests received	• **B** GET /?action=mwb_wgm_preview_mail&attachment=shell.php → 404 • **B** GET /wp-json/wc/store/v1/cart → 200 • **B** GET / → 200 • **B** GET /?action=dnd-wc-upload-file&name=shell.php → 404 • **B** GET /cart/ → 200
Files changed	• no files written
Database activity	questions +1818, select +1715, insert +13, update +5, delete +7

pool 1 unowned spare

Windows seen	none (0 requests)
Requests received	• none
Files changed	• no files written
Database activity	questions +589, select +554, insert +3, update +2, delete +2

Pool registry at export time

Growing normally – the host has room, so every new attacker gets a pool of their own and a spare is built after each assignment.
Ready pools: 3 (1, 2, 3). Free (unowned) spares: 0. Attackers per pool: pool 1: 1, pool 2: 2, pool 3: 1. Round-robin reuse assignments so far: 8.